COS341 Group Project (Group 8): SPL Compiler Front-End & Code Generator

A lexical analyzer, LL(1) top-down parser, syntax tree visualizer, semantic analyzer, and target code generator for the Simple Programming Language (SPL), built for COS341.

Members

Member Name	Student Number	Role
Christopher Adolph	23535548	Speaker
Dewald Colesky	23536030	Developer
Heinrich Romer	23538181	Developer
Jonty Honey	23536862	Developer

Declarations

1st Written Declaration: Submission Category

*We, the undersigned members of Group 8, hereby declare that this submission is made in the **"1st Class"** category for maximally 6/6 Points (including Semantics Analysis, Type Checking, and the generation of executable target code).*

- **Christopher Adolph** (23535548) -> 
- **Dewald Colesky** (23536030) -> 
- **Heinrich Romer** (23538181) ->

- **Jonty Honey** (23536862) -> 

2nd Written Declaration: Agreement with ZIP Contents

We, the undersigned members of Group 8, hereby declare that we are fully aware of the whole contents of the submitted ***`group-8.zip`*** *file and that we* ***agree*** *with the submission of the ZIP file in this form.*

- **Christopher Adolph** (23535548) -> 
- **Dewald Colesky** (23536030) -> 
- **Heinrich Romer** (23538181) ->
- **Jonty Honey** (23536862) -> 

What this project does

- **Phase 1:** Lexing, parsing, and syntax tree generation (writes `tree.xml`)
- **Phase 2a:** Semantic analysis, including scope resolution, symbol table construction, and name/function rule checking
- **Phase 2b:** Type analysis, ensuring operations are only performed on compatible types (e.g., no adding booleans, proper handling of `mod`/`div`)
- **Phase 3 / 1st Class:** Target code generation, translating SPL source files into human-legible **C** code (`<filename>.c`), compiled and tested using

standard `gcc` environments

Requirements

- Python 3.10 or newer
- Graphviz (optional, only for the graph visualizer)
- PyInstaller (only to create an executable)

Build the executable

Install PyInstaller:

```
pip install pyinstaller
```

Build a single-file executable from `main.py`:

```
pyinstaller --onefile --name group-8 main.py
```

Usage

Executable usage

Run the executable (the program file must be next to the executable):

```
./group-8 <filename>
```

Developer usage

Run the parser, semantic analyzer, and code generator manually (the program file must be next to `main.py`):

```
python3 main.py <filename>
```

This runs all compiler phases and outputs `<filename>.c`.

Compiling and running the generated C code

```
gcc <filename>.c -o program
./program
```

Visualizing the syntax tree (optional)

Requires Graphviz:

```
python3 graph_gen.py
```

This converts `tree.xml` into `syntax_tree.png`.

Output

On success, this writes:

- `tree.xml` (the syntax tree in the required ID, CONTENTS, CHILDREN, PARENT format)
- `<filename>.c` (the generated target code)

On failure, it prints one of:

- `SYNTAX ERROR: ...` if the input does not match the SPL grammar
- `LEXICAL ERROR: ...` if the input contains an invalid token
- One or more `SEMANTIC ERROR: ...` lines if the input is syntactically valid but violates a naming, scope, or function rule

Important note about the end of input

The pseudo symbol `$` is NOT literally included at the end of a real test file. `$` is only used to talk about the grammar and the parser, and it is not an actual member symbol of the SPL language itself. Our lexer automatically produces an end of file `$` token once it reaches the end of the input, whether or not the file itself contains a literal `$` character.

Project structure

File	Owner	Purpose
`tokens.py`	shared	Shared Token class, keyword and symbol tables
`lexer.py`	shared	Tokenizes SPL source into a stream of Tokens
`parser.py`	shared	Recursive descent LL(1) parser, builds the syntax tree and writes `tree.xml`
`graph_gen.py`	shared	Renders `tree.xml` as a `syntax_tree.png` image using Graphviz
`tree_crawl.py`	Colesky	Walks the syntax tree, assigns scope levels, builds the scope hierarchy
`symbol_table.py`	Jonty	Builds the symbol table from the scope tree, assigns unique internal (`sys###`) names

File	Owner	Purpose
`variable_rules.py`	Heinrich	Checks variable declaration and usage rules: ancestor lookup, no duplicates, no parameter masking
`function_rules.py`	Christopher	Checks function rules: name uniqueness, strictly hierarchic call resolution, recursion detection
`code_generator.py`	Christopher	Translates the checked AST and symbol table into human-legible C source code (`<filename>.c`)
`semantic_check.py`	shared	Combines all semantic rule checks into one entry point

Grammar

```
SPL_PROG  -> P $
P         -> V_DECL : F_DECL : ALGO
V_DECL    -> epsilon | USER-DEFINED-NAME V_DECL
F_DECL    -> epsilon | F_TYPE F_DECL
F_TYPE    -> void NAME ( V_DECL ) { P return }
           | num  NAME ( V_DECL ) { P return ( TERM ) }
ALGO      -> epsilon | INSTR ; ALGO
INSTR     -> USER-DEFINED-NAME INSTRTAIL | print OUTP | nop
           | comment STRING | BRANCH | LOOP
INSTRTAIL -> ( INPUT ) | = TERM
TERM      -> USER-DEFINED-NAME TERMTAIL | NUM
           | mod/add/sub/mul/div ( TERM TERM ) | neg ( TERM )
BRANCH    -> if BOOL then { ALGO } else { ALGO }
BOOL      -> not(BOOL) | and/or(BOOL BOOL) | eq/larger/lesser(TERM TERM)
LOOP      -> COND BOOL do { ALGO } | do { ALGO } COND BOOL
COND      -> while | until
```

The grammar is left factored for LL(1) parsing. `INSTRTAIL` and `TERMTAIL` resolve the shared `USER-DEFINED-NAME` prefix with one token of lookahead.

Semantic rules implemented

Variables: A declared name must exist in the usage's own scope or an ancestor scope, with the nearest declaration winning. No two declarations of the same kind are allowed at one scope level. A function's local variables may not mask its own parameters.

Functions: No duplicate function names are allowed at a scope level. Function calls are strictly hierarchic, meaning a call must resolve within the caller's own immediate scope, never an ancestor's. This one rule structurally blocks both direct and indirect recursion, along with sideways calls between sibling functions' private helpers.

Code generation & target environment notes

- **Target code language:** Standard C (`<filename>.c`), structured to be human-legible for inspection
- **Preferred run-test environment:** Fully compliant with standard C compilers (`gcc`, specifically tested against gcc 15+)
- **Submission tier:** Submitted under the "1st Class" category (incorporating full semantic analysis, type checking, and target code generation)

Status

☒ Phase 1: Lexer, Parser, Syntax Tree (`tree.xml`)
☒ Phase 2a: Scope analysis, Symbol Table, Variable and Function semantic rules
☒ Phase 2b: Type Analysis
☒ Phase 3: Target Code Generation (`<filename>.c`) compatible with gcc
☒ Executable build and packaging for submission